

[グラフ理論](#) [トーナメント](#) [強連結成分](#) [ソート](#) [ハミルトンパス](#) [ハミルトンサイクル](#)

トーナメント

[解説動画はこちら](#)です。

1. 概要

任意の相異なる 2 頂点 u, v について, $u \rightarrow v$ または $v \rightarrow u$ のどちらか一方が必ず存在するような単純有向グラフを**トーナメント**といいます. これは N 人が総当たり戦をしたときの勝敗を, 「勝った人から負けた人へ辺を張る」ことで表したものだと考えることができます.

定義は非常に簡潔ですが, トーナメントには一般の有向グラフには見られない強い構造が現れます. 例えば, 強連結成分同士の間には自然な順序があり, 各頂点の入次数だけから強連結成分分解が決定できてしまいます. またトーナメントにはハミルトンパスが必ず存在します.

本講座ではこのような, トーナメントについて成り立つ基礎的な性質を解説します.

2. 前提知識

入次数, 出次数, 強連結成分, DAG などの有向グラフの用語を仮定します. [グラフ理論用語集](#) を参照してください.

なお, 強連結成分分解のアルゴリズムは本講座では不要です.

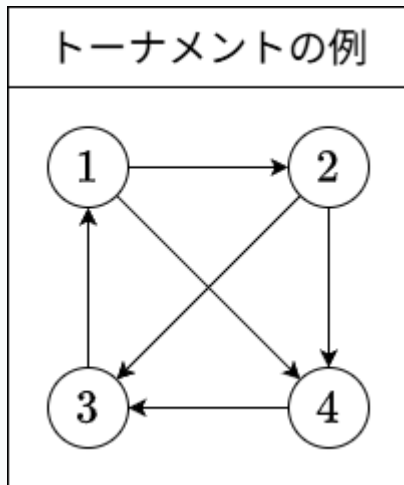
また, 6.2 節のみ, ソートアルゴリズムに関する知識を踏まえた解説をしています. (スキップ可能です.)

3. トーナメント

トーナメントの定義（グラフ理論用語集）を復習します。

【定義 1】

単純有向グラフであって、任意の相異なる 2 頂点 u, v に対して $u \rightarrow v$ または $v \rightarrow u$ のうちちょうど一方の辺が存在するものを**トーナメント**（tournament）といいます。



トーナメントは N 人が総当たり戦をした場合の勝敗結果をグラフ理論的に表したものだといえます。辺 $u \rightarrow v$ が「 u が v に勝った」という関係を表すと考えるとよいでしょう。

総当たり戦の結果が $N \times N$ の表として表されることが多いのと同様に、トーナメントは**隣接行列**として入出力されることが多いです。有向グラフの隣接行列とは、 $N \times N$ 行列であって、 (i, j) 成分が有向辺 $i \rightarrow j$ の個数と等しいものをいいます。例えば上の図のトーナメントの隣接行列は次のようになります。

$$\begin{pmatrix} 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 1 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{pmatrix}$$

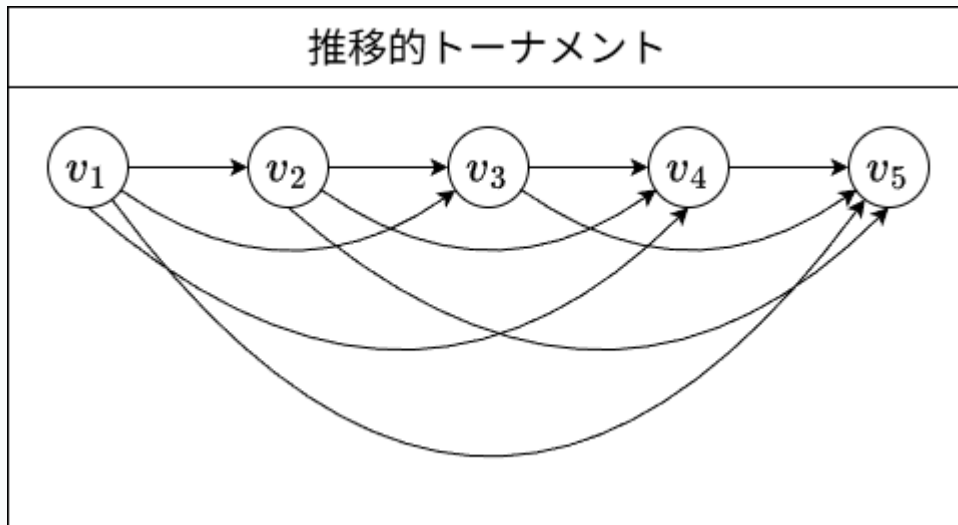
本講座の以下の部分では、「辺 $i \rightarrow j$ が存在する」ことを単に「 $i \rightarrow j$ である」と書くことにします。

4. 推移的トーナメント

トーナメントのうち、ある意味で最も極端な例が、本節で扱う**推移的トーナメント**です。これは、力関係がはっきりと順序付けられている N 人の総当たり戦の結果に対応します。

【定義 2】

N 頂点のトーナメント T が**推移的** (transitive) であるとは, N 頂点を適当な順に $v_1, v_2, \dots, v_N$ とすることで, 任意の $i < j$ に対して $v_i \rightarrow v_j$ であるようにできることをいう.



推移的トーナメントにはいくつかの同値な特徴付けがあります.

【定理 3】

トーナメント T について, 次は同値である.

1. T は推移的である.
2. T は DAG である.
3. T は長さ 3 のサイクルを含まない.
4. 任意の相異なる頂点 u, v, w に対して, $u \rightarrow v$ かつ $v \rightarrow w$ ならば, $u \rightarrow w$ である (推移律).

1 を仮定して 2 を示します. 頂点が $1, \dots, N$ で, $i < j$ ならば $i \rightarrow j$ であるようなトーナメントが DAG であることを示せばよいです. このトーナメントのウォークの頂点列 $i_1, i_2, \dots, i_k$ について $i_1 < i_2 < \dots < i_k$ が成り立つので, $k \geq 2$ について $i_1 = i_k$ となることはありません. したがってサイクルが存在しないため, DAG です.

2 が成り立つならば 3 が成り立つことは DAG の定義から明らかです.

3 を仮定して 4 を示します. $u \rightarrow v, v \rightarrow w$ であるとし. このとき $w \rightarrow u$ であるとする. 長さ 3 のサイクル $u \rightarrow v, v \rightarrow w, w \rightarrow u$ ができてしまいます. したがって $w \rightarrow u$ ではないので, T がトーナメントであることから $u \rightarrow w$ です.

最後に 4 (推移律) を仮定して 1 を示します. 頂点 v に対して $N^+(v)$ を, $v \rightarrow w$ であるような w 全体とします. 推移律より $u \rightarrow v$ であるとき, $N^+(v) \cup \{v\} \subseteq N^+(u)$ が成り立ちます.

$N^+(v)$ の要素数 $|N^+(v)|$ について T の頂点全体を降順ソートし, $v_1, v_2, \dots, v_N$ とします (要素数が等しいときには適当な順でソートします).

$i < j$ に対して辺 $v_j \rightarrow v_i$ が存在すると, $N^+(v_i) \cup \{v_i\} \subseteq N^+(v_j)$ より $|N^+(v_i)| < |N^+(v_j)|$ となり, 降順ソートしていたことに矛盾します. したがって $v_j \rightarrow v_i$ ではありません. T がトーナメントであることから $v_i \rightarrow v_j$ です.

したがってこの頂点順は 【定義 2】 の条件を満たし, T は推移的です. ■

▶ 参考：三すくみ

5. トーナメントの強連結成分分解

5.1. 強連結成分間の辺

4 節で見たように, トーナメントが DAG である場合には, 頂点に上手く番号をつけると辺の向きは番号の大小関係と一致することが分かります.

同じようにトーナメントの強連結成分についても, 上手く番号をつけると, 成分の間の辺の向きが番号の大小関係と一致することが分かります.

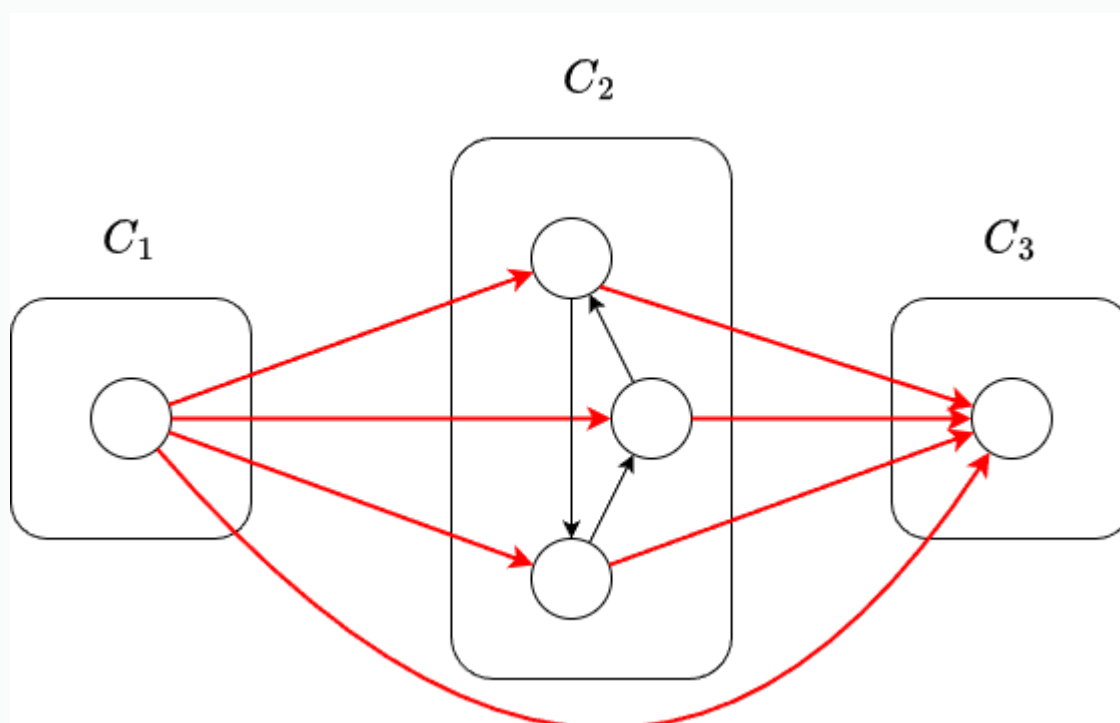
【定理 4】

トーナメント T が K 個の強連結成分を持つとする. これらの強連結成分を適当な順に $C_1, C_2, \dots, C_K$ とすることで, 次が成り立つようにできる: $i < j$ ならば, 任意の $u \in C_i$ と $v \in C_j$ に対して $u \rightarrow v$.

各 C_i から頂点をひとつ選び、 c_i としましょう。頂点集合 $S = \{c_1, c_2, \dots, c_K\}$ による誘導部分グラフ $T[S]$ はトーナメントであり DAG なので、推移的トーナメントです。よって適当に添字を付けなおすことで、 $i < j$ ならば $c_i \rightarrow c_j$ であるようにできます。

このとき $i < j$ について $u \in C_i$, $v \in C_j$ とすると、 u から v へのウォークが存在します（ u から c_i へのウォーク、 c_i から c_j への辺、 c_j から v へのウォークを結合したもの）。このことと u, v が相異なる強連結成分の頂点であることから、 $v \rightarrow u$ ではありません。よって $u \rightarrow v$ です。 ■

本講座の以下の部分では、強連結成分 $C_1, C_2, \dots, C_K$ と書いた場合、**【定理 4】** のように添字付けられているものとします。



5.2. 強連結成分分解とカット

【定義 5】

T をトーナメント（あるいはより一般の有向グラフ）、 S を T の頂点からなる集合とすると、有向辺の集合 $\delta^+(S), \delta^-(S)$ を

$$\begin{aligned}\delta^+(S) &= \{v \rightarrow w \mid v \in S, w \notin S\}, \\ \delta^-(S) &= \{v \rightarrow w \mid v \notin S, w \in S\}\end{aligned}$$

により定義する.

【定理 6】

T をトーナメント, $C_1, C_2, \dots, C_K$ をその強連結成分とする. このとき, 頂点集合 S について次は同値である.

1. ある k ($0 \leq k \leq K$) に対して $S = C_1 \cup C_2 \cup \dots \cup C_k$ が成り立つ. (ただし, $k = 0$ のとき右辺は $\emptyset$ であるとする.)
2. $\delta^-(S) = \emptyset$.

【定理 4】より 1 が成り立つならば 2 も成り立ちます. 逆を示します.

$S = \emptyset$ のときは $k = 0$ が条件を満たします. $S \neq \emptyset$ のとき $S \cap C_k \neq \emptyset$ であるような最大の整数 k をとり, 条件を満たすことを示します.

k のとり方から $S \subseteq C_1 \cup C_2 \cup \dots \cup C_k$ は明らかです. 逆の包含を示します.

$v \in C_i$ ($1 \leq i \leq k$) とすると, C_k のすべての頂点に対して v からのウォークが存在します. したがって v から S の頂点へのウォークが存在します. すると $\delta^-(S) = \emptyset$ の仮定から $v \in S$ が成り立ちます. ■

5.3. 入次数列と強連結成分分解

5.1, 5.2 節で見たように, トーナメントの強連結成分はかなり強い構造があります. このことから, トーナメントの強連結成分分解も一般の有向グラフの場合よりも簡単に求められる場合がよくあります. (ただしすべての辺が入力で与えられるようなトーナメントでは入力に $O(N^2)$ 時間がかかるので, そのような場合には計算量オーダーの意味で一般の有向グラフに対するアルゴリズムを上回るわけではありません.)

特に, 何らかの基準によって頂点列をソートして, 【定理 6】のような境界 S を見つけるという手法が有効になることがあります. ここではそのような一例として, トーナメントにおいては頂点の入次数だけから強連結成分分解が求まることを示します.

頂点 v の入次数を $\deg^-(v)$ と書きます.

【補題 7】

$i < j$ かつ $u \in C_i, v \in C_j$ ならば $\deg^-(u) < \deg^-(v)$ が成り立つ.

【定理 4】 から明らかです.

【定理 8】

トーナメント T の頂点を $v_1, v_2, \dots, v_N$ とし, 入次数が広義単調増加である, つまり

$$\deg^-(v_1) \leq \deg^-(v_2) \leq \dots \leq \deg^-(v_N)$$

が成り立つとする. このとき次が成り立つ.

1. S が 【定理 6】 の条件を満たすならば, ある k に対して $S = \{v_1, v_2, \dots, v_k\}$ が成り立つ.
2. $S = \{v_1, v_2, \dots, v_k\}$ が 【定理 6】 の条件を満たすことと, $\sum_{v \in S} \deg^-(v) = \binom{k}{2}$ が成り立つことは同値である.

1 は補題より明らかです.

2 を示します. $\sum_{v \in S} \deg^-(v)$ は, S の頂点を終点とする辺の個数です. これは次の和です.

- 始点, 終点ともに S に含まれる辺の個数.
- 始点が S に含まれず, 終点が S に含まれる辺の個数. つまり $|\delta^-(S)|$.

トーナメントにおいて前者の辺の個数は $\binom{k}{2}$ です. よって $\sum_{v \in S} \deg^-(v) = \binom{k}{2}$ は $\delta^-(S) = \emptyset$ と同値です. ■

【系 9】

トーナメントの強連結成分分解は, 各頂点の入次数だけから定まる.

【定理 8】 から明らかです. ■

6. ハミルトンパス

6.1. ハミルトンパスの存在証明

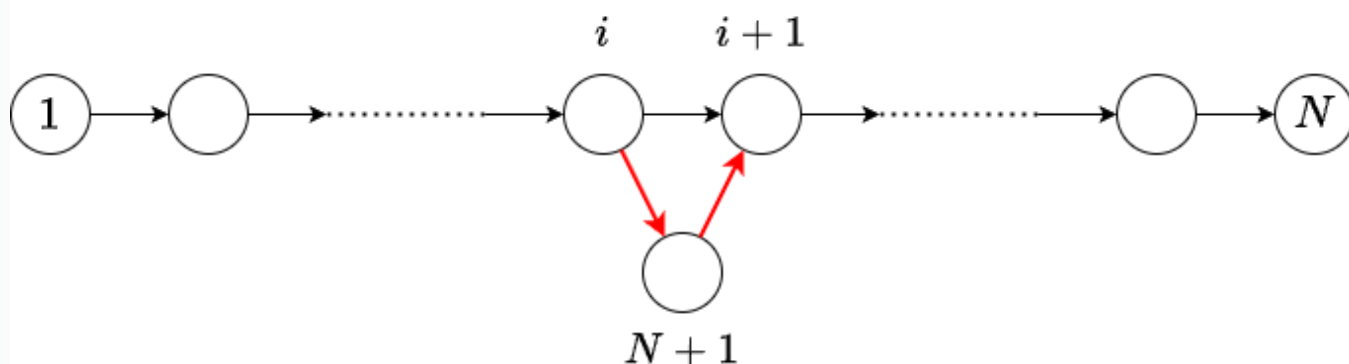
【定理 10】(Rédei の定理)

頂点数 N が 1 以上のトーナメントにはハミルトンパスが存在する。

N に関する帰納法により示します。 N を正整数とし、 N について成り立つとして、 $N + 1$ の場合に示します。

トーナメントの頂点を $1, 2, \dots, N, N + 1$ とします。 頂点集合 $\{1, 2, \dots, N\}$ に関する誘導部分グラフについて帰納法の仮定を用いることで、これら N 頂点すべてを通るパスが存在します。 必要があれば頂点番号を入れ替えることで、頂点列 $(1, 2, \dots, N)$ がパスであるとしてよいです。

- $N + 1 \rightarrow 1$ ならば、頂点列 $(N + 1, 1, 2, \dots, N)$ がハミルトンパスになります。
- $N \rightarrow N + 1$ ならば、頂点列 $(1, 2, \dots, N, N + 1)$ がハミルトンパスになります。
- このどちらでもないとき、 $i \rightarrow N + 1, N + 1 \rightarrow i + 1$ となる i ($1 \leq i \leq N - 1$) が存在します (例えば $i \rightarrow N + 1$ となる最大の i が条件を満たします)。 このとき、 $(1, 2, \dots, i, N + 1, i + 1, \dots, N)$ がハミルトンパスになります。



以上により示されました。 ■

アルゴリズム, 計算量について

上の証明はそのままハミルトンパスを構築するアルゴリズムになります。 $i \rightarrow N + 1, N + 1 \rightarrow i + 1$ となる i を見つけることは、二分探索により $O(\log N)$ 時間でできます。

見つけた $i, i + 1$ の間に $N + 1$ を挿入する部分を単純な配列操作で行えば、計算量は $O(N^2)$ となります。挿入に適切なデータ構造を用いれば $O(N \log N)$ 時間にもなりますが、 $O(N \log N)$ 時間での構築に拘る場合には 6.2 節の方法の方が実装が簡潔だと思います。

6.2. 参考：比較ソートとの関係

与えられるトーナメント T が推移的な場合には、ハミルトンパスを求める問題は、「 $i \rightarrow j$ であるときに $i < j$ と見なす」という順序によって頂点をソートする問題と等価です。

このことを踏まえて改めて、 T が推移的な場合に 6.1 節のアルゴリズムを見直すと、ソート列の適切な位置に新しい頂点をひとつずつ挿入していくという**挿入ソート**と同様の手順になっていることが確認できます。言い換えれば、6.1 節のアルゴリズムは、トーナメントの辺の向きを、**推移的でない場合にも無理やり大小関係と考える**、挿入ソートを行ったものです。計算量についても挿入ソートと同様になっています。

この考え方は、挿入ソートに限らず、多くの比較ソート（クイックソート、マージソートなど）にも当てはまります。つまり、トーナメントの辺の向きを大小関係と考える比較ソートを実行すると、得られた出力列 $v_1, v_2, \dots, v_N$ はハミルトンパスになります。これらのアルゴリズムでは、最終的に隣り合う 2 要素 v_i, v_{i+1} はアルゴリズムの過程で比較され、その比較結果と整合する順序で並ぶためです。

例えば、クイックソートを元に考えることで、**【定理 10】** は次のように証明することもできます。

- 適当な頂点 v （**ピボット**）をとり、 $x \rightarrow v$ となる x 全体を L 、 $v \rightarrow x$ となる x 全体を R とする。
- 頂点集合 L, R について再帰的にハミルトンパスを求める。 L のハミルトンパス、 v 、 R のハミルトンパスをこの順に結合することで全体のハミルトンパスを得る。

計算量についてもクイックソートと同様で、ピボットを乱択することで期待時間計算量 $O(N \log N)$ が達成されます。（計算量解析の議論もおおよそ同様ですが、トーナメントの入次数、出次数に関する議論が必要です。）

また、マージソートを元に考えることで、乱択を使わずに時間計算量 $O(N \log N)$ でハミルトンパスを見つけることもできます。

7. ハミルトンサイクル

トーナメントにはハミルトンパスが存在することを示しましたが、強連結かつ頂点数 3 以上の場合には、より強くハミルトンサイクルも存在します。

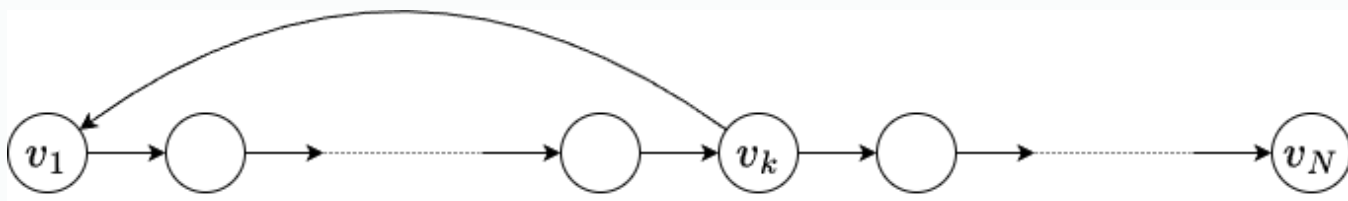
【定理 11】（Camion の定理）

頂点数 N が 3 以上の強連結なトーナメントにはハミルトンサイクルが存在する。

【定理 10】の証明と似た、サイクルに頂点やパスを挿入していくというタイプの議論により証明します。

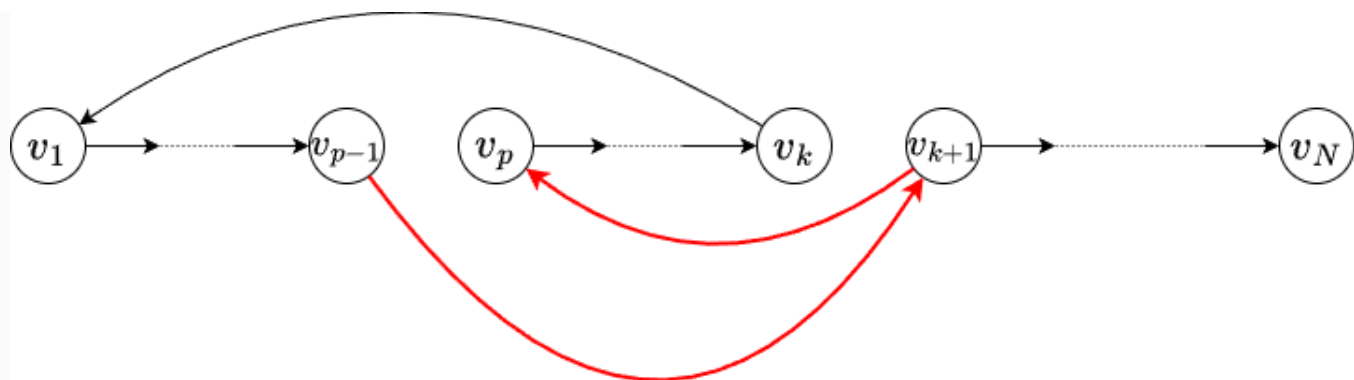
まず、ハミルトンパス $v_1 \rightarrow v_2 \rightarrow \cdots \rightarrow v_N$ をひとつとります。 $N \geq 3$ かつ強連結であることから、辺 $v_k \rightarrow v_1$ が存在します。このような k をひとつとります。結果、次のような状況が得られています。

$$v_1 \rightarrow v_2 \rightarrow \cdots \rightarrow v_N, \quad v_k \rightarrow v_1$$

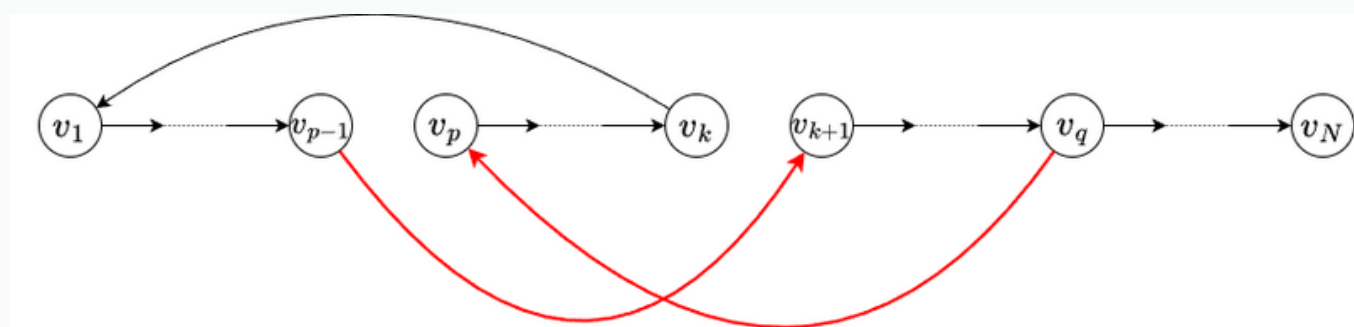


$k < N$ であるとして、適当に頂点番号をつけなおすことなどでこの状況が成り立つ k を大きくできることを示せばよいです。

まず、 $v_{k+1} \rightarrow v_1$ である場合には、頂点番号をそのまま k を $k+1$ にとりかえてもこの状況が維持されるのでよいです。以下 $v_1 \rightarrow v_{k+1}$ であるとします。 $v_{k+1} \rightarrow v_p$ となる $p \leq k$ が存在するならば、そのうち最小のものをとれば、 $2 \leq p$, $v_{p-1} \rightarrow v_{k+1}$, $v_{k+1} \rightarrow v_p$ が成り立ちます。したがって、 v_{p-1} と v_p の間に v_{k+1} を挿入することで、 k を大きくすることができます。形式的には、 $v_p, \dots, v_k, v_1, \dots, v_{p-1}, v_{k+1}$ を改めて $v_1, v_2, \dots, v_{k+1}$ ととりなおし、 k を $k+1$ にとりかえればよいです。



次に、任意の p ($p \leq k$) について $v_p \rightarrow v_{k+1}$ であると仮定します。強連結性の仮定から、 $v_q \rightarrow v_p$ となる $p \leq k, k+2 \leq q$ が存在します。 $p = 1$ であれば、頂点番号をそのまま k を q にとりかえることができます。 $2 \leq p$ であるとし、 $v_{p-1} \rightarrow v_{k+1}, v_q \rightarrow v_p$ であることから、 v_{p-1} と v_p の間にパス $v_{k+1} \rightarrow \dots \rightarrow v_q$ を挿入することができます。形式的には $v_p, \dots, v_k, v_1, \dots, v_{p-1}, v_{k+1}, \dots, v_q$ を改めて $v_1, v_2, \dots, v_q$ ととりなおし、 k を q にとりかえればよいです。 ■



アルゴリズム，計算量について

上の証明はそのままハミルトンサイクルを構築するアルゴリズムになります。

v_q を見つけるときに、手前側の頂点から順に調べることで、同じ組 (i, j) 間での辺の存在判定回数が $O(1)$ 回になり、適切に実装すれば、 $O(N^2)$ 時間でハミルトンサイクルを構築することができます。

8. 2-king

最後に、トーナメントに関するこれまでとは少し方向性の異なる定理として、2-king と呼ばれる頂点の存在を紹介します。

【定理 12】（Landau の定理）

空でないトーナメントについて、ある頂点 v が存在して次が成り立つ：任意の頂点 x について、 v から x への距離は 2 以下である。つまり、 $x \neq v$ ならば次のどちらかが成り立つ。

- $v \rightarrow x$.
- ある頂点 y に対して $v \rightarrow y, y \rightarrow x$.

v が出次数最大の頂点であるときに、この条件が成り立つことを示します。反例となる頂点 x が存在したと仮定します。

このとき、 $v \rightarrow y, y \rightarrow x$ となる y が存在しないことから、 $v \rightarrow y$ ならば $x \rightarrow y$ が成り立ちます。つまり $N^+(v) \subseteq N^+(x)$ が成り立ちます。さらに $v \rightarrow x$ ではないことから $x \rightarrow v$ なので、 $N^+(v) \cup \{v\} \subseteq N^+(x)$ が成り立ちます。したがって v の出次数よりも x の出次数の方が大きいことになってしまいますが、これは v が出次数最大の頂点であることに矛盾します。 ■

9. 関連問題

1. <https://codeforces.com/contest/117/problem/C>
2. <https://codeforces.com/contest/1498/problem/E>
3. <https://codeforces.com/contest/1779/problem/E>
4. https://atcoder.jp/contests/arc215/tasks/arc215_c
5. https://atcoder.jp/contests/snuke21/tasks/snuke21_e
6. https://atcoder.jp/contests/arc163/tasks/arc163_d
7. https://atcoder.jp/contests/pakencamp-2025-day1/tasks/pakencamp_2025_day1_p
8. <https://yukicoder.me/problems/no/2085>
9. <https://codeforces.com/contest/412/problem/D>
10. <https://codeforces.com/problemset/problem/323/B>
11. <https://qoj.ac/contest/1784/problem/9246>
12. https://atcoder.jp/contests/joisp2024/tasks/joisp2024_l
13. <https://codeforces.com/contest/1481/problem/D>
14. <https://codeforces.com/contest/1514/problem/E>

15. <https://codeforces.com/problemset/problem/1268/D>

16. https://atcoder.jp/contests/agc046/tasks/agc046_f

▶ 問題について

10. まとめ

本講座では、トーナメントについて成り立つ基本的な定理を解説しました。

一般の有向グラフではハミルトンパスやハミルトンサイクルの存在判定は非常に難しい問題ですが、トーナメントはそれらの存在判定や構築が比較的簡潔に行えるグラフの代表例です。

また、強連結成分分解の構造やハミルトンパス・ハミルトンサイクルの構成では、トーナメントにおける辺の向きが順序を表すかのように振る舞う場合があることが感じられたと思います。このことは、トーナメントが N 人で総当たり戦をした場合の勝敗結果を表したものだと考えれば、自然に感じられるかもしれません。

11. 参考文献

1. Wikipedia. Tournament (graph theory).
[https://en.wikipedia.org/wiki/Tournament_\(graph_theory\)](https://en.wikipedia.org/wiki/Tournament_(graph_theory)). (閲覧日: 2026-04-01).
2. Yannis Manoussakis. A linear-time algorithm for finding Hamiltonian cycles in tournaments. Discrete Applied Mathematics. Vol. 36. pp. 199–201. 1992.
DOI: 10.1016/0166-218X(92)90233-Z. [https://doi.org/10.1016/0166-218X\(92\)90233-Z](https://doi.org/10.1016/0166-218X(92)90233-Z).
3. Amotz Bar-Noy, Joseph Naor. Sorting, Minimal Feedback Sets, and Hamilton Paths in Tournaments. SIAM Journal on Discrete Mathematics. Vol. 3. No. 1. pp. 7–20. 1990. DOI: 10.1137/0403002.
<https://doi.org/10.1137/0403002>.



AtCoderは、世界最高峰の競技プログラミングサイトです。リアルタイムのオンラインコンテストで競い合うことや、5,000以上の過去問にいつでもチャレンジすることができます。



[ライセンス表記](#)

AtCoderについて

[AtCoderとは？](#)

[競技プログラミングとは？](#)

[レーティングのしくみ](#)

[アクセスガイド](#)

[お知らせ](#) [🔗](#)

資料請求

[AtCoder](#) [🔗](#)

[AtCoderJobs](#) [🔗](#)

各サービス

[AtCoder](#) [🔗](#)

[AtCoderJobs](#) [🔗](#)

[アルゴリズム実技検定](#) [🔗](#)

[AtCoderCareerDesign](#) [🔗](#)

AtCoder株式会社について

[会社概要](#)

[FAQ](#) [🔗](#)

[プライバシーポリシー](#) [🔗](#)

[利用規約](#) [🔗](#)

